

Android Project Architecture & Codebase

Summary Report

Project Name: Beacon

Target Auditor Audience: External Technical Auditor

Date: March 2026

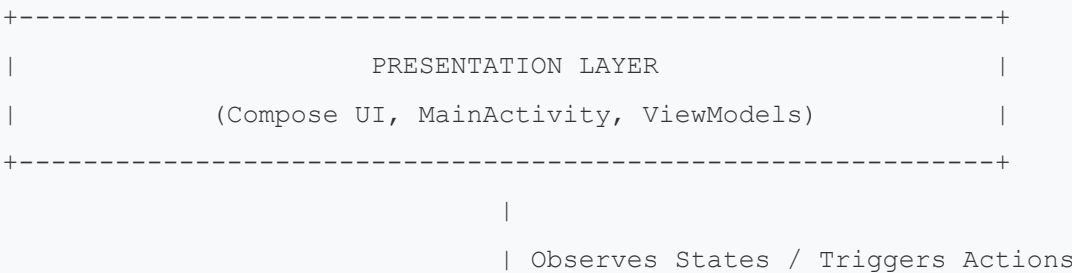
1. Architectural Overview

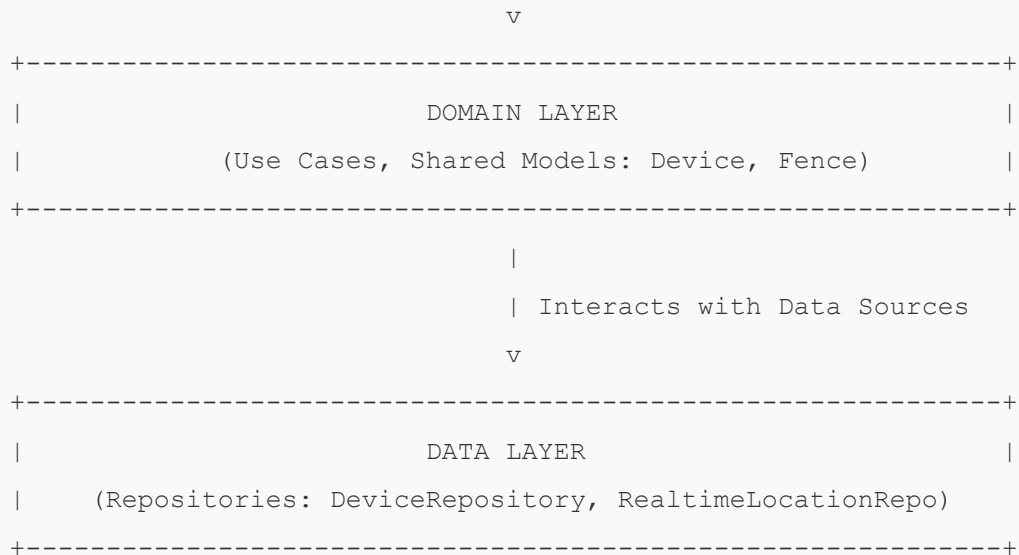
The **Beacon** project is structured as a multi-module Android application composed of three distinct Gradle modules:

- **:app (Admin Application)**: Displays a map tracking interface, allows managing paired tracker devices, and configures and visualizes geofences.
- **:tracker (Device Tracking Client)**: Runs on target tracking devices to periodically acquire GPS coordinates and synchronize them to Firebase.
- **:shared (Common Libraries & Models)**: Shares domain models (e.g., Device, Fence, Group), serialization mappers, and constants between the Admin and Tracker apps.

Design Pattern: Clean Architecture & MVVM (Model-View-ViewModel)

Each application module (:app and :tracker) adheres to **MVVM** principles combined with **Clean Architecture** patterns to enforce unidirectional data flow (UDF) and decoupling of layers:





- **Presentation Layer (UI & ViewModels):** Powered by **Jetpack Compose** for a modern, declarative, and reactive UI. ViewModels manage screen-level state and interact with the Repository layer.
- **Domain Layer:** Encapsulated in the `:shared` module, specifying immutable data structures and schema mappings to guarantee compatibility.
- **Data Layer (Repositories):** Abstracts data operations. ViewModels interact with repositories (e.g., `DeviceRepository`, `FenceRepository`) rather than accessing Cloud Firestore or Firebase Realtime Database directly.

2. Feature & Functional Matrix

Admin App (`:app`) Features

- **Real-time Multi-Device Tracking:** Plots live positions of paired devices on an interactive map (rendered via OpenStreetMap's `osmdroid` library).
- **Interactive Geofencing:**
 - **Zone / Checkpoint Fences:** Supports creating boundary fences (Zones) and single-point checkpoints.
 - **Move / Reposition:** Drag-and-drop red map markers to physically move/reposition existing fences.
 - **Custom Notifications:** Generates alerts when tracked devices enter or exit designated zones.

- **Flexible Device Configuration:**
 - Set interval-based or high-frequency "Live" tracking rates.
 - Configure thresholds (e.g., Low Battery alerts, Offline notifications).
 - Toggle Smart Battery Saving options.
- **SOS Notifications & Heads-Up Alerts:**
 - Configured with high-importance channels (`NotificationManager.IMPORTANCE_HIGH`) and `setFullScreenIntent`.
 - Uses window attributes (`showOnLockScreen`, `showWhenLocked`, `turnScreenOn`) in `AndroidManifest.xml` to immediately wake the device and display critical distress overlays even when locked.

Tracker App (: tracker) Features

- **Persistent Background Tracking:** Employs a robust `ForegroundService` with custom sticky status notifications to fetch GPS locations in the background.
- **Onboarding Setup & Permission Guard:** Displays a setup screen guiding users to grant precise location permissions, background location tracking permissions, notification permissions, and bypass "aggressive" battery optimization settings.
- **Instant SOS Distress Trigger:** Provides an interactive emergency SOS button. When activated, it forces continuous tracking updates and sends alarms to the Admin application.
- **Smart Battery Optimization:** Monitors motion utilizing hardware sensors (like `SignificantMotionSensor`) to place tracking updates in dormant mode when stationary, dramatically extending battery life.

3. Data Layer & Database Schema

The system uses a hybrid Firebase database strategy: **Cloud Firestore** for configuration, user metadata, and geofence boundaries, and **Firebase Realtime Database (RTDB)** for high-frequency location streaming.

A. Cloud Firestore Schemas

1. devices (Collection)

Defines device settings, ownership, and tracking modes.

```
{
  "deviceId": "STRING (Unique identifier)",
  "device_id": "STRING (Fallback copy)",
  "ownerId": "STRING (Admin's User UID)",
  "owner_id": "STRING (Fallback copy)",
  "deviceName": "STRING (Friendly name)",
  "device_name": "STRING (Fallback copy)",
  "trackerAuthUid": "STRING (Authentication reference)",
  "status": "online | offline",
  "commandMode": "off | interval | live",
  "command_mode": "STRING (Fallback copy)",
  "intervalSeconds": "INT (Update frequency in seconds)",
  "autoRevertSeconds": "INT (Duration before live mode reverts)",
  "isEmergencyMode": "BOOLEAN (Indicates distress state)",
  "batterySavingEnabled": "BOOLEAN (Stationary sleep control)",
  "stationaryIntervalMinutes": "INT (Update interval when resting)",
  "alertThresholds": {
    "lowBatteryPercent": "INT (Battery alert trigger percent)",
    "offlineThresholdMinutes": "INT (Offline timeout trigger minutes)"
  },
  "sosFallbackPhone": "STRING (Phone number for emergency contact)",
  "commandTimestamp": "LONG (Unix timestamp of last settings update)"
}
```

2. fences (Collection)

Configures localized geographical boundaries.

```
{
  "id": "STRING (Auto-generated UUID)",
  "name": "STRING (Fence identifier, e.g., 'Home')",
  "centerLat": "DOUBLE (Geographic latitude)",
  "centerLng": "DOUBLE (Geographic longitude)",
  "radiusMeters": "DOUBLE (Boundary circle radius)",
}
```

```
"type": "zone | checkpoint",
"alertOnEnter": "BOOLEAN",
"alertOnExit": "BOOLEAN"
}
```

B. Firebase Realtime Database (RTDB) Schemas

/live_locations

Hosts real-time, low-latency telemetry updates for paired active trackers.

```
{
  "deviceId123": {
    "latitude": 37.7749,
    "longitude": -122.4194,
    "status": "online | offline",
    "batteryLevel": 85,
    "lastSeen": 1710123456000
  }
}
```

4. Key Classes & Modules

Module: :app (Admin Client)

- **MainActivity.kt**: The entry point of the Admin application. Houses navigation flows and hosts the background monitoring listener for device alerts.
- **MapScreen.kt**: Orchestrates the main map UI using `osmdroid`. Manages map overlays, live updates for markers, and controls interactive geofence repositioning.
- **DeviceSettingsSheet.kt**: A bottom sheet dialog managing configurations, low-power policies, offline limits, and manual sync commands for individual trackers.
- **AdminSosService.kt**: A foreground monitoring service in the Admin client. Observes Firestore changes for active SOS statuses, triggering full-screen alerts and sound alarms when an emergency arises.

Module: :tracker (Tracking Endpoint)

- **LocationTrackingService.kt:** The heart of the tracker module. A background service implementing fine location loops, managing tracking intervals dynamically, listening to device settings adjustments, and monitoring motion state changes.
- **OnboardingScreen.kt:** Handles startup configuration checklist. Prompts runtime permissions requests (ACCESS_FINE_LOCATION, POST_NOTIFICATIONS) and provides seamless intent actions guiding users to white-list battery optimizations.
- **BatteryMonitorService.kt:** Monitors platform battery broadcast channels safely and registers updates without memory leaks.

5. Security & Robustness Measures

- **Safe Receivers:** Implements dynamic registration checking targeting `Build.VERSION_CODES.TIRAMISU` and higher, safely configuring broadcast receivers with `Context.RECEIVER_NOT_EXPORTED` flags to block external app-level exploits.
- **Targeted App Intent Launches:** Guarantees inter-app navigation remains private by defining explicit target packages on implicit system Intents (e.g., `.setPackage("com.google.android.apps.maps")`), protecting the app's components from hijacking.